Rev 2.0

EC551: Advanced Digital Design with Verilog and FPGAs
Fall 2026 | Professor Douglas Densmore
Programming Assignment 2 – "Hardware Synthesis"

- **Revision:** 2.0 (Enhanced HLS Edition)
- **Due Date:** November 10 (Four-day extension included due to initial release delay)
- **Submission:** Public GitHub Repository containing source code, generated artifacts, and the required README.

Overview & Objective

The objective of this assignment is to build a simplified **High-Level Synthesis (HLS) Tool** and a corresponding hardware runtime environment. You will write a software program that takes high-level arithmetic equations, parses them, creates an Intermediate Representation (IR) Directed Acyclic Graph (DAG), performs resource-constrained scheduling, and automatically generates synthesized Verilog RTL (Datapath + Control FSM). Finally, you will deploy and test your synthesized design on an FPGA.

Detailed Specifications

1. High-Level Input Parsing

- **Input Format:** Read from a text file containing a series of equations, one equation per line. Each equation consists of two arguments using either addition (+), multiplication (*), or subtraction (-).
    - *Example:*

      X = A + B
      Y = X + A
      Z = X - B
      Q = B * Z

- **Assumptions:**
    - You can assume there will be no more than **20 lines** of equations.
    - You do **not** need to deal with results that evaluate to negative numbers.

2. Single Assignment Form (SSA)

- Convert the parsed equations into **Single Assignment / Single Static Assignment (SSA)** form by ensuring variables are dynamically renamed upon re-assignment (e.g., turning repeated variable updates into uniquely indexed versions).
- **Feature Requirement:** Your software tool must be able to export this SSA result as a text file upon request (e.g., via a command-line flag --export-ssa).

3. Directed Acyclic Graph (DAG) & Scheduling

- Construct a **DAG** for the single-assignment statements where nodes represent arithmetic operators and edges represent data dependencies.

- Implement a scheduler that can specify and output a schedule based on any of the following modes:
    - **ASAP (As Soon As Possible):** Unconstrained latency.
    - **Constrained ASAP:** ASAP scheduling bounded by a user-specified maximum number of adders, subtractors, and/or multipliers.
    - **ALAP (As Late As Possible):** Unconstrained latency working backward from a bound.
    - **Constrained ALAP:** ALAP scheduling bounded by hardware resource limits.
    - **Custom Schedule:** A custom scheduling strategy of your own design.

4. Datapath and Sequencer Logic Generation

- **Datapath Construction:** Automatically generate a Mux- and Register-based design.
    - Registers must sit at the cycle boundaries determined by your scheduled DAG.
    - Multiplexers must be utilized to minimize total hardware units and/or respect the hardware resource constraints defined by your scheduler.
- **Control Unit / Sequencer:** Generate finite state machine (FSM) control logic to manage Mux select lines and register write-enables across cycle boundaries.

5. Verilog Functional Unit Blocks

- Create basic modular Verilog functional blocks for **adders**, **subtractors**, and **multipliers**.
- **Bit-width Selection:** Choose a bit-width that allows you to cleanly demonstrate operations on your physical FPGA (e.g., think about how you are going to input values).
- **Demo Initialization:** Pick a bit-width that allows initial values for core variables to be fed via physical FPGA switches/buttons, with remaining static/unassigned variables hardcoded for demo convenience.

6. Composite Verilog Synthesis & RTL Integration

- Combine your generated control/datapath structures with your primitive functional units to create a complete composite Verilog system:
    - Create a dedicated Verilog module for the generated synthesizer output.
    - Instantiate the precise number of adder, subtractor, and multiplier modules required by the schedule.
    - Insert Mux-like routing logic and pipeline registers alongside the functional units.
    - Create a top-level module binding everything together.
    - Write comprehensive testbenches (testbench.v) to simulate and verify the system behavior prior to board deployment.
- Rev 2.0 addition – allow for the “clocking” of your system to be an external button so you can “step” the data through the system in a controlled way for the demo
- Rev 2.0 addition – make sure to have the observability of your functional units be selected (e.g. have a switch control which adder, multipler, or substrator is being shown on the output)

7. FPGA Deployment & Verification

- Synthesize, place, route, and flash your entire system onto the FPGA target board.
- **I/O Integration:** Implement a mechanism to display output values clearly (e.g., routing results to **7-segment displays**, LEDs, or serial output) to verify real-time functionality.

8. README & Demo Plan

- Provide a comprehensive README.md containing a structured **30-minute Demo Plan**.
- The demo plan must:
    - Fit strictly within a 30-minute window and demonstrate every aspect of your software and hardware system.
    - Allow **Professor Densmore** to write custom equations on the spot, compile them via your tool, run the FPGA, and verify the computed outputs.
    - Accommodate **three distinct live test runs** directed by the professor.

Submission Guidelines

Submit a link to a public **GitHub repository** containing:

1. **Source Code:** Well-organized software (Python/C++/Rust) and Verilog files.
2. **README File:**
    - Explanation of repository organization and directory structure.
    - Short description of your synthesis tool architecture.
    - Instructions on how to read, build, and run the code.
    - Your complete 30-minute demo plan.

Note: In addition to the code submission, sign up for a presentation slot to demo your code live in front of Professor Densmore. Grading will evaluate both programmatic correctness/robustness and the design quality of your live demo plan.